Cambridge (CIE) A Level Computer Science

Your notes

Program Testing & Maintenance

Contents

- Faults & Errors
- Testing Methods
- Maintenance

© 2025 Save My Exams, Ltd.

Your notes

Error types

What are the common types of error?

- Designing algorithms is a skill that must be developed and when designing algorithms, **mistakes and issues will occur**
- **Trace tables** can also help to find any kind of error in a program or algorithm
- There are three main categories of errors that when designing algorithms a programmer must be able to identify & fix, they are:
  - **Syntax errors**
  - **Logic errors**
  - **Runtime errors**

Syntax errors

What is a syntax error?

- A syntax error is **an error that breaks the grammatical rules of a programming language** and **stops it from running**
- Examples of syntax errors are:
  - **Typos and spelling errors**
  - **Missing or extra brackets or quotes**
  - **Misplaced or missing semicolons**
  - **Invalid variable or function names**
  - **Incorrect use of operators**
  - **Incorrectly nested loops & blocks of code**

Pseudocode – with syntax errors

```
FUNCTION generate_username(first_name STRING, last_name STRING) RETURNS STRING
   DECLARE username : STRING
   username = SUBSTRING(first_name, 1, 1) + SUBSTRING(last_name, 1, 1) + SUBSTRING(last_name, 1, 3)
   RETURN username
ENDFUNCTION

PROCEDURE main()
   DECLARE first_name : STRING
   DECLARE last_name : STRING
   DECLARE username : STRING

   OUTPUT "Enter your first name: "
```

```
   INPUT first_name

   OUTPUT "Enter your last name: "
   INPUT last_name

   username ← generate_username[first_name, last_name]

   OUTPUT "Here's a suggested username:" + username
ENDPROCEDURE

// Main program
CALL Main
```

Pseudocode – without syntax errors

```
FUNCTION generate_username(first_name : STRING, last_name : STRING) RETURNS STRING
   DECLARE username : STRING
   username ← SUBSTRING(first_name, 1, 1) + SUBSTRING(last_name, 1, 1) + SUBSTRING(last_name, 1, 3)
   RETURN username
ENDFUNCTION

PROCEDURE main()
   DECLARE first_name : STRING
   DECLARE last_name : STRING
   DECLARE username : STRING

   OUTPUT "Enter your first name: "
   INPUT first_name

   OUTPUT "Enter your last name: "
   INPUT last_name

   username ← generate_username(first_name, last_name)

   OUTPUT "Here's a suggested username: ", username
ENDPROCEDURE

// Main program
CALL main()
```

Syntax errors

- FUNCTION generate_username(first_name STRING...)
  - **Missing colon** : between parameter name and type
  - first_name : STRING and last_name : STRING
- username = SUBSTRING(...)
  - **Uses =** instead of correct assignment operator ←
  - Should be username ← ...
- username ← generate_username[first_name, last_name]
  - **Incorrect use of square brackets []** instead of parentheses () for function call

Your notes

- Should be generate_username(first_name, last_name)
- CALL Main
  - **Incorrect capitalisation and missing parentheses** — procedure name is case-sensitive
  - Should be CALL main()

Your notes

Logic errors

What is a logic error?

- A logic error is where **incorrect code is used that causes the program to run**, but **produces an incorrect output or result**
- Logic errors can be difficult to identify by the person who wrote the program, so one method of finding them is to use '**Trace Tables**'
- Examples of logic errors are:
  - **Incorrect use of operators (< and >)**
  - **Logical operator confusion (AND for OR)**
  - **Looping one extra time**
  - **Indexing arrays incorrectly (arrays indexing starts from 0)**
  - **Using variables before they are assigned**
  - **Infinite loops**

Pseudocode

```
FUNCTION calculate_area(length : REAL, width : REAL) RETURNS REAL
  // Checks for valid dimensions and calculates area
  IF length > 0 OR width > 0 THEN
    DECLARE area : REAL
    area ← width - length
    RETURN area
  ELSE
    OUTPUT "Length and width must be positive values."
  ENDIF
ENDFUNCTION

PROCEDURE main()
  DECLARE length : REAL
  DECLARE width : REAL
  DECLARE area : REAL

  OUTPUT "Enter the length of the rectangle: "
  INPUT length

  OUTPUT "Enter the width of the rectangle: "
  INPUT width
```

```
    area ← calculate_area(length, width)

    OUTPUT "The area of the rectangle is approximately ", area, " square units."
ENDPROCEDURE

// Main program
CALL main()
```

Logic errors

Line	Error	Correction
IF length > 0 OR width > 0 THEN	The check allows one value to be negative, which is not valid	Should be IF length > 0 AND width > 0 THEN
area ← width - length	Incorrect calculation – subtracts instead of multiplying to find the area	Should be area ← length * width

Runtime errors

What is a runtime error?

- A runtime error is where **an error causes a program to crash**
- Examples of runtime errors are:
  - **Dividing a number by 0**
  - **Index out of the range of an array**
  - **Unable to read or write a drive**

Pseudocode

```
FUNCTION calculate_area(length : REAL, width : REAL) RETURNS REAL
   // Calculates area of a rectangle after input validation
   IF length < 0 OR width < 0 THEN
      OUTPUT "Length and width must be positive values."
   ENDIF

   DECLARE area : REAL
   area ← length * width
   RETURN area
ENDFUNCTION

PROCEDURE main()
   DECLARE length : REAL
   DECLARE width : REAL
   DECLARE area : REAL

   OUTPUT "Enter the length of the rectangle: "
   INPUT length
```

Your notes

```
   OUTPUT "Enter the width of the rectangle: "
   INPUT width

   area ← calculate_area(length, width)

   OUTPUT "The area of the rectangle is approximately ", area, " square units."
ENDPROCEDURE

// Main program
CALL main()
```

Your notes

Runtime error

Line	Error	Correction
area ← calculate_area(length)	Only one parameter is passed instead of two – will crash at runtime	Should be area ← calculate_area(length, width)

Worked Example

Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type. **[5]**

Statement	Error
Result ← 2 & 4	
SubString ← MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result ← Status AND INT(x/2)	
Message ← "Done" + LENGTH(MyString)	

Answer

Statement	Error

Result ← 2 & 4	Should be arithmetic operator (not &) **OR** 2 and 4 should be CHAR / STRING
SubString ← MID("pseudocode", 4, 1)	NO ERROR
IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator **OR** Condition incorrect
Result ← Status AND INT(x/2)	INT(X/2) doesn't evaluate to a Boolean value / incorrect operator
Message ← "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number

Your notes

Your notes

Testing methods

What are the different methods of testing?

- Testing takes place during the **testing stage** of the **program development life cycle**, once the program has been written and compiled
  - **Syntax errors** are usually detected during compilation
  - Testing focuses on checking for **logic errors**, **unexpected behaviour**, and **operational integrity**
  - A detailed **test plan**, outlining test cases and expected results, should be created during the **analysis stage**

Sample test plan

- Scenario: A program accepts a user's age between 0 and 120 inclusive

Test No.	Description	Input	Expected output	Type of test data	Actual outcome
1	Valid age within accepted range	25	"Age accepted"	Normal	
2	Input is not a number	"twenty"	"Invalid input: enter a number"	Abnormal	
3	Age above the maximum allowed	130	"Age out of range"	Extreme	
4	Age exactly at the upper boundary	120	"Age accepted"	Boundary	
5	Age exactly at the lower boundary	0	"Age accepted"	Boundary	
6	Age just below lower boundary	-1	"Age out of range"	Extreme	
7	Blank input	(blank)	"Please enter your age"	Abnormal	

Common methods of testing

© 2025 Save My Exams, Ltd.

Testing method	Description
Dry run	**Manually trace through** the code (e.g. on paper) to predict output and track variables
Walkthrough	Step-by-step review of the code **with others** to identify issues early
White-box testing	Tests the **internal logic** and code structure; the tester knows how the program works
Black-box testing	Tests the **inputs and expected outputs** without knowing the internal workings
Integration testing	Checks that different modules or components **work correctly together**
Alpha testing	Performed **in-house** by the developers during early testing
Beta testing	Carried out by **external users** in real-world environments
Acceptance testing	Final check to ensure the program meets the original **client requirements**
Stub testing	Uses temporary modules (stubs) to **simulate missing components** during early testing

Your notes

Choosing suitable test data

What is suitable test data?

- Suitable test data is **specially chosen to test the functionality** of a program or design
- Developers or test-users would pick **a selection of test data** from the following categories
  - **Normal**
  - **Abnormal**
  - **Extreme**
  - **Boundary**
- The results would be **compared to the expected results** to check if the algorithm/program works as intended
- The results would be **stored in the test plan**
- Each category is explained within the context of a simple pseudocode program below

Pseudocode

DECLARE name : STRING
DECLARE age : INTEGER
 	
OUTPUT "What is your name? "
INPUT name
 	
OUTPUT "How old are you? "
INPUT age
 	
IF age >= 12 AND age <= 18 THEN
  OUTPUT "Welcome, " + name + "! Your age is accepted."
ELSE
  OUTPUT "Sorry, " + name + ". Your age is not accepted."
ENDIF

Your notes

Normal data

- Normal test data is data that **should be accepted** in the program
- An example would be a user entering their age as 16 into the age field of the program

Abnormal data

- Abnormal test data is data that is the **wrong data type**
- An example would be a user entering their age as "F" into the age field of the program

Extreme data

- Extreme test data is the **maximum** and **minimum** values of normal data that are **accepted** by the system
- An example would be a user entering their age as 18 or 12 into the age field of the program

Boundary data

- Boundary test data is similar to extreme data except that the values **on either side of** the **maximum** and **minimum** values are tested
- The largest and smallest **unacceptable values**
- An example would be a user entering their age as 11 or 19 into the age field of the program

Selecting suitable test data

Type of Test	Input	Expected Output
Normal	14	Accepted
Normal	16	Accepted

© 2025 Save My Exams, Ltd.

Extreme	12	Accepted
Extreme	18	Accepted
Abnormal	H	Rejected
Abnormal	@	Rejected
Boundary	11	Rejected
Boundary	19	Rejected

Your notes

© 2025 Save My Exams, Ltd.

Your notes

Types of maintenance

What is program maintenance?

- Program maintenance is the process of updating or improving a program **after it has been delivered** to the user
- Unlike physical equipment, programs **don't wear out**, but they may need to be changed due to **errors**, **changing requirements**, or **new technology**

Why maintenance is needed

- To **fix errors that were missed** during testing
- To **improve performance** based on user feedback
- To **adapt the software** to new uses or platforms

Types of program maintenance

Type	Purpose	Example
Corrective	Fixes bugs or errors found during real-world use	Fixing a bug that causes the program to crash when special characters are entered
Perfective	Improves performance or adds small enhancements	Replacing a loading screen with a progress bar to give better user feedback
Adaptive	Modifies the program to support new environments or requirements	Modifying the program to work on a tablet instead of just a desktop computer

© 2025 Save My Exams, Ltd.